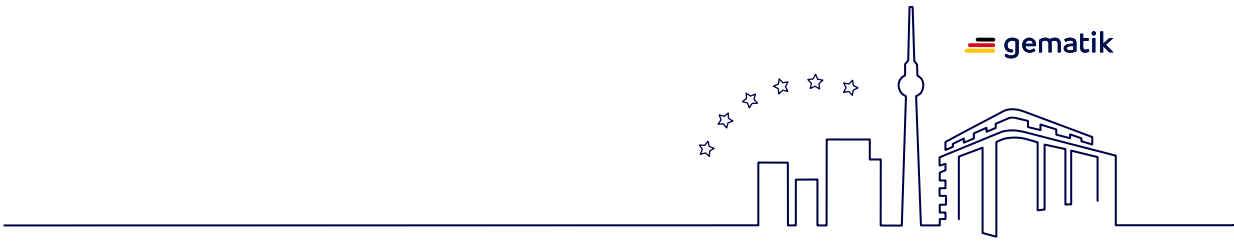
Erpione
Documentation

gematik GmbH

Version 1.0.0, 2026-11-06



Contents

1. Introduction . . . 2
   1.1. Requirements . . . 2
      1.1.1. Reference to E-Rezept . . . 2
   1.2. Quality Goals . . . 2
2. Architecture Constraints . . . 4
3. Context and Scope . . . 5
   3.1. Business Context . . . 5
   3.2. Technical Context . . . 5
4. Solution Strategy . . . 7
5. Building Block View . . . 8
   5.1. Whitebox Overall System . . . 9
6. Runtime View . . . 11
   6.1. Scope and Purpose . . . 11
   6.2. Recurring Runtime Concepts . . . 11
   6.3. Typical Runtime Flows . . . 11
      6.3.1. Prescription creation . . . 12
      6.3.2. Prescription claim, dispensation and close . . . 13
      6.3.3. Manual exploratory workflow via Web UI . . . 13
      6.3.4. Automated API workflow . . . 14
   6.4. Sequence Diagrams . . . 14
7. Deployment View . . . 16
   7.1. Infrastructure Level 1 . . . 16
   7.2. Infrastructure Level 2 . . . 16
      7.2.1. Resource Server Container . . . 17
      7.2.2. Authentication Server Container . . . 17
      7.2.3. Reverse Proxy Container . . . 17
      7.2.4. Web UI Container . . . 17
8. Cross-cutting Concepts . . . 18
   8.1. Hermes – FHIR Abstraction . . . 18
      8.1.1. Hermes Data Model . . . 18
      8.1.2. Translation . . . 18
      8.1.3. Faker Capability . . . 19
   8.2. Actor Model . . . 19
      8.2.1. Actor Roles . . . 19
      8.2.2. Actor Resolution . . . 20
      8.2.3. Execution as an Actor . . . 20
   8.3. Local Persistence . . . 21
   8.4. Error Handling . . . 21
   8.5. TI Connectivity . . . 22
9. Build, Release & Operations . . . 24

9.1. Overview 24
9.2. Prerequisites 24
9.2.1. System Requirements 24
9.2.2. Build Tools 24
9.2.3. Supported Operating Systems 25
9.3. Build Process 25
9.3.1. Repository Structure 25
9.3.2. Building Java/Kotlin Modules 25
9.3.3. Building Frontend 26
9.3.4. Building Docker Images 26
9.4. Configuration & Environment Variables 27
9.4.1. Environment Files 27
9.4.2. Environment Variables by Service 28
9.4.3. Spring Boot Profiles 29
9.5. Secrets Management 30
9.5.1. Development Secrets 30
9.5.2. E-Rezept Fachdienst Credentials (`application-secrets.yaml`) 30
9.5.3. Production Secrets 32
9.6. Deployment Procedures 32
9.6.1. Prerequisites Checklist 32
9.6.2. Local Development Workflow 33
9.6.3. Production Deployment 34
9.7. Health Checks & Validation 34
9.7.1. Service Health Endpoints 34
9.7.2. Manual Health Check Script 35
9.7.3. Database Verification 35
9.7.4. API Verification 36
9.8. Logging & Monitoring 36
9.8.1. Application Logs 36
9.8.2. Log Locations 36
9.8.3. Metrics & Monitoring 36
9.9. Troubleshooting 37
9.9.1. Common Issues 37
Appendix A: Release Notes 39



This document describes the current *Erpione* architecture. The documentation of the predecessor **PrimSys** is no longer available as rendered HTML. But you can still find the original AsciiDoc source files in the `adocs/legacy_primsys/` folder of this repository.

1. Introduction

This document describes the architecture of Erpione, a modern open source e-Health platform simulator. Erpione's primary goal is to provide a comprehensive simulation environment for E-Rezept client systems. This enables vendors to test their applications by simulating the missing bits of the E-Rezept workflow, such as the PVS, AVS, and NCPeH systems. Erpione allows vendors to focus on their own application logic without needing access to other real complementary systems, which are often unavailable or impractical for testing purposes.

1.1. Requirements

Erpione must provide the following core capabilities to support its business goals:

- Simulate the behaviour and interactions of complementary systems in the E-Rezept workflow. These systems include PVS (Praxisverwaltungssystem), AVS (Apothekenverwaltungssystem), and NCPeH (National Contact Point for eHealth).
- End-to-end testing of E-Rezept client systems must be possible even when real complementary systems are not available.
- It is essential to abstract the complexity of these systems so that vendors can develop and test their own solutions efficiently.

The primary users are vendors developing client systems for E-Rezept, ePA, and other solutions that interact with the E-Rezept backend service.

1.1.1. Reference to E-Rezept

Erpione is designed as a 'Primärsystem-Simulator' based on the E-Rezept system specifications. Erpione's requirements and functionalities align with the following official E-Rezept documents and resources:

- Systemspezifisches Konzept E-Rezept
- Spezifikation E-Rezept-Fachdienst
- Spezifikation Datenmodell E-Rezept
- Implementierungsleitfaden Primärsysteme – E-Rezept

These documents serve as the authoritative basis for the simulation logic and interoperability provided by Erpione. For detailed technical and process requirements, please refer to the official E-Rezept documentation at gemspec.gematik.de (or the current official source).

1.2. Quality Goals

Prio	Quality Goal	Description
1	Ease of Use	Erpione must provide a seamless and intuitive experience for vendors, abstracting away the complexity of the E-Rezept ecosystem. The simulator must handle the intricate details of the complementary systems. This allows users to focus on developing and testing their own client logic without deep knowledge of the complementary systems.
2	End-to-End and Exploratory Testing	The architecture must minimise the overhead and friction involved in testing end-to-end workflows. Erpione must enable rapid feedback cycles and support both automated E2E and manual exploratory testing scenarios.
3	Maintainability and Adaptability	Erpione should be easy to maintain and extend, allowing for quick adaptation to changes in the E-Rezept specifications or the needs of different vendors. The architecture must support modular updates and clear separation of concerns.

2. Architecture Constraints

Erpione shall be:

- plattform-independent and should run in containerized environments
- designed to provide a REST API for integration with other test systems and tools
- usable without real hardware components. Examples of these components include Konnektors and eHealth-Kartenlesegeräte, which are typically required for E-Rezept workflows. By simulating these components, Erpione allows vendors to test their applications without needing access to physical hardware, which can be costly and difficult to obtain for testing purposes.
- modular and extensible to allow for future extensions and adaptations to different use cases
- built on gematik's shared test foundations rather than re-implementing the E-Rezept domain. Erpione reuses the E-Rezept E2E Testsuite, which in turn is based on the Bricks framework (B²riC²s). As a consequence Erpione inherits the Screenplay Pattern (actors, abilities, use cases) and the FHIR handling conventions defined there; see the Bricks User Manual for the underlying concepts and Building Block View for how the layers fit together.

3. Context and Scope

3.1. Business Context

Erpione acts as a "Primärsystem Simulator" for the E-Rezept ecosystem. Its primary function is to assist vendors in developing and testing client systems for the E-Rezept, ePA, and associated services by simulating the key interactions with the E-Rezept backend.

The business context includes:

- Simulation of LEI (Leistungserbringende Institution):
  - PVS (Praxisverwaltungssystem)
  - AVS (Apothekenverwaltungssystem)
  - KTR (Kostenträger)
  - NCPeH (National Contact Point for eHealth)
- Providing test data for end-to-end testing of E-Rezept workflows.
- Providing test data randomization capabilities to support exploratory testing and robustness testing.

The business context does not include:

- Simulation of FdV (Frontend des Versicherten).
- Integration of real hardware components e.g. Konnektor.

Erpione uses an actor-based approach to model interactions. This concept is heavily inspired by the Screenplay Pattern with its Actor Model, which emphasizes using actors to represent different roles and responsibilities in a system. In this context, actors can be considered entities that perform actions and interact with the E-Rezept Fachdienst, thus facilitating the modeling of complex interactions in a clear, modular way. All Erpione actors are preconfigured with their own identity, derived from SMC-B, HBA and some additional configurations. A more detailed description of Erpione actors and their runtime handling is provided in Actor Model.

3.2. Technical Context

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9oLLKsFsZ0GlUTNMFJI7g2qbLWg8RIKJIhKLHlniOT5QEKc4x18
xCXsw4Ar_xrsue13ajKsN63UynlFcvam4l8OmzX1wi61P1pzpRyZr9jKVYBq3GVxBMSP3QSiLiWxWE6NGVzpT64m7WAle0PxSKhLbO1NH
OV32BUB5kXPr9BE10HOm5bQ2t-tXJtF6xf3WzuJ2YO-_JJq-
dqZDVfe_BWL4dEP9R7Z42g1HpW0r9c3O1aFe8EmGExSVvhuCnkVP7ANOxExTcpXJfyQyBcxM28LAxLdltwwrvnL5LrLG7-HW_xY-
fGNFkI8T_UGGuxO4vfYZXCKn2mBZMUrLaHVzSUfDI6AVkF1uARmRCgIb56WSg6DGnm2YUas1kUbagTaDUSQZHOaoMAi1cuylXeiAtKZxJ
yJJMEYK7sytxyP3A0fopL7fw0ip3r6BZqGpcYY61sJP9roac3WNN4K4fADfQ1irurffA5kbdRHkcZO9gVO_T3F-mFGaAaJfrFwGPN_-
EWRqH-CKjWh9pfdSbs_EqPhIs5peTfHrEifdXddLpbcR3TcMywst2zNPo-0fqHreAls_402s5IRfg_k5enECn-
D2bki5IZoLIAAnSbs9otYgit3BII5WKKtE7i-
TaelBzNMxf6xLvwrR3c7sdox8hIPfjQoSPYYyD0oI7gHQnmpHMQvMomWmbG_qwy52jAQ6eMrbdB1AbLSOTORhQooVikG_Q7QKNtiOi9oL
jHjIIZZqLh0leTceQ_rxBW9cPlc8RSxXnEa4daus8NABY4rfJ5-
kGRu4pJPInNDBmXlEF80_0Gas54YtfncgT0jKHcuBlfMRPJwJ_a7H00rFW00" />

Erpione exposes its simulation capabilities via well-defined technical interfaces:

- **REST APIs**: For interaction with the simulated client systems.

- **Web-based User Interface**: For manual and exploratory testing.

Mapping Input/Output to Channels:

- Domain-specific behaviours (e.g., prescription creation, claim, status updates) are mapped to REST endpoints.
- The Erpione REST endpoints are primarily designed to communicate using the Hermes Data Model, a simplified JSON representation of the E-Rezept domain data which abstracts away the complexities of the underlying FHIR-based E-Rezept specifications.
- For certain behaviours (e.g., creating prescriptions), Erpione may also support the use of FHIR resources as input. These resources are then directly used for the communication with the E-Rezept Fachdienst.

4. Solution Strategy

Erpione is primarily implemented in Java using the Spring Boot framework, which provides a robust foundation for building modular, maintainable, and production-grade services. The data model is defined in Kotlin, chosen for its seamless interoperability with Java, enhanced flexibility, and strong null-safety features, which help prevent common programming errors and improve code quality.

The system architecture follows the Backend-for-Frontend (BFF) pattern. Authentication is managed via Keycloak, which is integrated into the BFF layer to provide secure, standards-based identity management.

Deployment is handled using Docker Compose, targeting on-premise environments. This enables straightforward setup, reproducibility, and isolation of all required services, making it easy for vendors and testers to run Erpione in their own infrastructure.

These technology and architectural decisions directly support Erpione's key quality goals: high operability and ease of use, reduced overhead for end-to-end testing, and maintainability. By abstracting the complexity of the E-Rezept ecosystem and providing a flexible, secure, and easy-to-deploy simulator, Erpione enables vendors to focus on their own development and testing tasks without being burdened by the details of complementary systems.

5. Building Block View

This section provides an overview of Erpione's static architecture by describing its main building blocks, their responsibilities, and how they interact. Erpione itself is based on a layered and modular architecture which is heavily inspired by the Screenplay Pattern. The building blocks are designed to be cohesive, loosely coupled, and easily testable.

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9nzaijEmp0GHVV-
2WjsI8424eegAseQ6XRbfRRv03UPjbPT9x9TeGhuToPsceS4P1FFdElhyKsOiIS94Q7uyDoMcdwwDQNNyQ2aTChp_LQNHrKafKHo2H3UF
upkDfsYr0Ne0MtODpcxPWR-DGt33V_RrDFMj6TzD91p0wrrRVpKDqxJD9o53NqM2ff-
SXkcer5ZnMwQF2R6ScsF1qd8JXH0ZGKDfJrLGCYuliEOSbCVgYbZTHr5PEmISy1dnvXx8oBdyIvGInDSNTIoB26ama7v5acy5b9O0SQXU
EgkeY6tN6rboq-U-lLGC5kyqqGAKDPfiFJLK3KlbO8zZkL4yxVNtqJB0zu-
vxhmmonVuiKgYHUFIu9nzsUgAYboRaMfA4QDQLWm_lWuMxbhQnhe86eI6TCAzARK1rGGtt8qIpfa25P-
JxR2jb2OFVwRz4FONSrO49m3G0oAhoL-5UzsLMzhsQu-fQdO5sxhCXcqknG6xTwroGHKWR_33olRy-00" />

Erpione does not implement the E-Rezept domain itself but builds on two gematik libraries, each adding one layer of abstraction:

- **Bricks (B²riC²s)** is the generic foundation. The Bricks framework provides reusable building bricks for maintainable test suites: the Screenplay core with actors, abilities and use cases shown in the diagram above, FHIR building bricks for profile-safe resource handling, and software abstractions for smartcards (HBA, SMC-B) and the Konnektor. Its concepts are documented in the Bricks User Manual.
- **E-Rezept E2E Testsuite** is the domain layer built on Bricks. The E-Rezept E2E Testsuite contributes everything specific to the E-Rezept: the FHIR profiles of KBV, gematik and DAV, the client for the E-Rezept Fachdienst and the use cases such as prescribe, accept, dispense and close. It is the heart of the system and the place where the core functionality dealing with the Fachdienst is built and maintained.
- **Erpione** exposes these capabilities as a service. It adds the REST API and the Web UI, the Hermes abstraction and the actor stage with preconfigured identities, so that the testsuite's actors can be driven interactively or by external test systems.

<object type="image/svg+xml"
data="https://plantuml.gematik.solutions/plantuml/svg/U9oLLjjkcq0GVUShfgHI7YfSagOEhLmhbzgLrKiisr4VegXQms2
lGW3jhXkbbt_lmgvXWSLIyn8pSywSsPdZDICkdXCSEo3_I2Wo1h_Bpm23oyOZm1K97pOiswNHTPR8p15Yy6RedyOr8cCHiaPMv_uw-
wgt70yM3OAOz1TLsQgeKNh7CIGSgz95uKzcuUbq6am2dVr2KzJnwvDWwlkwrEYrdiS55uI9tMFYE2zUvh8MsU2D_4_5Ctm0zoHsPNpDQF
XWHCzyju27HU7fbh8YTXJ7PoU-
xvhnguB7PTQ0leAMJ8VGmPlobUu2kOXtIIaT16_zE7PRgPRwdj3RW4cpiQnjp97HHnIePa1840ph6LJ9LYirgRUP9jNEj3Qqp1bDDsKtx
n3NP-TkCtELPQ8gfy6zNTIKPM6JyfHjPhCBNGTSTMHN-xGAntAdtbhdEaunlZ1t9JBC8WmL_huDT-
hlHbMUgNMr59g6C3KYJyaa55FHr359DgbylugMM7lfXXn_b4CC2OkuHtBQb6lIQiv3bgRu86yeQvF_onea60jltDEAUS_qpAOgq5cF8ub
tDCeruHu81tSMpU6wV7RXoFVFXqFq0yCb4s-1lp0NC3cTm4fEdE-
eG0EhntU7B2_RiRkWJEzrUzPNOkfZzqomhUP5sLCw6FondhWZhT0bs3QzBg4QPwLemJ2wsLOWumGMQAalaSOUx8UlExKxGIwptfBwihlG
kKVqSdWPyTOov75rKN2rVtrDb61zXV1k7PqfH6jdi9ncQM45zGFhnrgDfX8Qar1Y5fFb2YxdCxVGcYpcivjlan_okLTWFugYiYT_e7_I4
5MZMWwMAkWseJkgKDzn3RUpLhAuSxo82DAiz556uAkqI-
8wFSBUVbtTxHy0jlTw06gQfGUl75WOmHZWVME2tLaeImxQfFSm6idta_7ua9RkhJKwYyOkxu4wnmKuof2dnm8OflAz2SIM27Z2955PYtZ
LSy_bs3kXYcZrUU7QD1C89GookFTugoq2tnA6aM4ErNYOfOBGbCidwPpL3Iy94SRo_Is2VCSmUPREXy-PI6WwSEIPeH1rd0lvKRw-
_GFv0uM2" > <img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/svg/U9oLLjjkcq0GVUShfgHI7YfSagOEhLmhbzgLrKiisr4VegXQms2l
GW3jhXkbbt_lmgvXWSLIyn8pSywSsPdZDICkdXCSEo3_I2Wo1h_Bpm23oyOZm1K97pOiswNHTPR8p15Yy6RedyOr8cCHiaPMv_uw-
wgt70yM3OAOz1TLsQgeKNh7CIGSgz95uKzcuUbq6am2dVr2KzJnwvDWwlkwrEYrdiS55uI9tMFYE2zUvh8MsU2D_4_5Ctm0zoHsPNpDQF
XWHCzyju27HU7fbh8YTXJ7PoU-
xvhnguB7PTQ0leAMJ8VGmPlobUu2kOXtIIaT16_zE7PRgPRwdj3RW4cpiQnjp97HHnIePa1840ph6LJ9LYirgRUP9jNEj3Qqp1bDDsKtx
n3NP-TkCtELPQ8gfy6zNTIKPM6JyfHjPhCBNGTSTMHN-xGAntAdtbhdEaunlZ1t9JBC8WmL_huDT-
hlHbMUgNMr59g6C3KYJyaa55FHr359DgbylugMM7lfXXn_b4CC2OkuHtBQb6lIQiv3bgRu86yeQvF_onea60jltDEAUS_qpAOgq5cF8ub
tDCeruHu81tSMpU6wV7RXoFVFXqFq0yCb4s-1lp0NC3cTm4fEdE-
eG0EhntU7B2_RiRkWJEzrUzPNOkfZzqomhUP5sLCw6FondhWZhT0bs3QzBg4QPwLemJ2wsLOWumGMQAalaSOUx8UlExKxGIwptfBwihlG

kKVqSdWPyTOov75rKN2rVtrDb61zXV1k7PqfH6jdi9ncQM45zGFhnrgDfX8Qar1Y5fFb2YxdCxVGcYpcivjlan_okLTWFugYiYT_e7_I4
5MZMWwMAkWseJkgKDzn3RUpLhAuSxo82DAiz556uAkqI-
8wFSBUVbtTxHy0jlTw06gQfGUl75WOmHZWVME2tLaeImxQfFSm6idta_7ua9RkhJKwYyOkxu4wnmKuof2dnm8OflAz2SIM27Z2955PYtZ
LSy_bs3kXYcZrUU7QD1C89GookFTugoq2tnA6aM4ErNYOfOBGbCidwPpL3Iy94SRo_Is2VCSmUPREXy-PI6WwSEIPeH1rd0lvKRw-
_GFv0uM2" /> </object>

Important Interfaces

The following interfaces are central to the interaction between Erpione's building blocks and with external systems:

- **REST API**: The main interface for client systems and the web frontend, exposing endpoints for prescription management, actor actions, and test data generation. Follows RESTful conventions and uses JSON (not FHIR) as the primary data format.
- **Data Model (Kotlin/Java Classes)**: Shared data structures used across the REST API, ensuring consistent serialization and deserialization of business objects. These classes form the **Hermes Data Model**, which abstracts away the complexities of the underlying FHIR while still supporting all necessary data for testing scenarios. The translation between the Hermes Data Model and FHIR is handled internally by the HermesService. Hermes and its capabilities are described in detail in Hermes.
- **FHIR Interface**: Used internally and externally for E-Rezept data exchange, based on the HL7 FHIR standard. Ensures compatibility with the E-Rezept backend and client systems.
- **Authentication & Authorization (OAuth2/OpenID Connect)**: Integrates with Keycloak to secure API endpoints and the frontend, supporting OAuth2 and OIDC flows for user and system authentication.
- **Persistence Layer**: Provides access to stored test data and simulation state via Spring Data JPA.

These interfaces are designed to be modular and extensible, supporting both automated and manual testing scenarios.

5.1. Whitebox Overall System

The following section details the internal structure of Erpione by breaking down the overall system into its main building blocks and illustrating their interactions.

<object type="image/svg+xml"
data="https://plantuml.gematik.solutions/plantuml/svg/U9o5bDjkr30GXk_p5AFbPX5DAI2XLHL5RRSDBIfGzV00RZBPDVL
wWZqfAgZlpYJnfaa9PIzMyHm-p_nZE_7yO32DG7ugPtBmk_q6s3qSBG6EL7wtSbJROab6FAyG5ny-
xhqldoB85UX6tk1xZBRK6uylGXUBKlqRsdbxw01TUyoLnnxTW3yDmLcMBKuMmNkkBGRxyjqYsziBgEHjqEF0it9SRqmKLK9IAuJPYQiqM
OGhTFSwnvdi5KNAMcB5ue3P9NfUacL7ng2RWVAGzqkV9BvosgxYMo9EqoZIbj6LAaS8v5RnG9VexYDDGqzlu9vq0GvBXtvzLbJNT8VMpr
y7xxdsd0JIcVMikMxAEbQiKbWXjtH_PajocxRWFlDRrIoLIKvH6Lxdok65etijkhUPVvldLzoq0bPjy0NAe8mMD5YF2Oz3DKxHRT077Vh
ccJvxIVikVzSF5Uv0kTPE5AVQvNqDJ9bOE_Iy2UX22z5W7B9DV37egDQcQRRnTFjraRUTlLbi4mhCgUXeIwEytsadZvQVzJQSTDzQaA8H
gEY7ycJyAX9DcBzOt93LUU-
QddsdUqbzs5ZUThoDuJdcGewid2YqNT0bLkIrLFqG0kCOq5LnRhHq2TASAb2CmwEghMQjZFw5GxX8Nv7rc5ocSfX_r79fvgsv_HnOjpdP
LjrMsIHBOI1pqEF_Qj-C7eXZg2H6RiAXdnzsBr-
RExYli5lGJnkdavHzI19NMxd6mupOwTMQdwLibTu7ZmPpqShHPfnci6oonfFP1tRAUgCOuQVcTHHDKIcUg3AQAdcoEDab226lbGXmWBQG
L-yFEWNwXW00" > <img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/svg/U9o5bDjkr30GXk_p5AFbPX5DAI2XLHL5RRSDBIfGzV00RZBPDVLw
WZqfAgZlpYJnfaa9PIzMyHm-p_nZE_7yO32DG7ugPtBmk_q6s3qSBG6EL7wtSbJROab6FAyG5ny-
xhqldoB85UX6tk1xZBRK6uylGXUBKlqRsdbxw01TUyoLnnxTW3yDmLcMBKuMmNkkBGRxyjqYsziBgEHjqEF0it9SRqmKLK9IAuJPYQiqM
OGhTFSwnvdi5KNAMcB5ue3P9NfUacL7ng2RWVAGzqkV9BvosgxYMo9EqoZIbj6LAaS8v5RnG9VexYDDGqzlu9vq0GvBXtvzLbJNT8VMpr
y7xxdsd0JIcVMikMxAEbQiKbWXjtH_PajocxRWFlDRrIoLIKvH6Lxdok65etijkhUPVvldLzoq0bPjy0NAe8mMD5YF2Oz3DKxHRT077Vh
ccJvxIVikVzSF5Uv0kTPE5AVQvNqDJ9bOE_Iy2UX22z5W7B9DV37egDQcQRRnTFjraRUTlLbi4mhCgUXeIwEytsadZvQVzJQSTDzQaA8H
gEY7ycJyAX9DcBzOt93LUU-

QddsdUqbzs5ZUThoDuJdcGewid2YqNT0bLkIrLFqG0kCOq5LnRhHq2TASAb2CmwEghMQjZFw5GxX8Nv7rc5ocSfX_r79fvgsv_HnOjpdP
LjrMsIHBOI1pqEF_Qj-C7eXZg2H6RiAXdnzsBr-
RExYli5lGJnkdavHzI19NMxd6mupOwTMQdwLibTu7ZmPpqShHPfnci6oonfFP1tRAUgCOuQVcTHHDKIcUg3AQAdcoEDab226lbGXmWBQG
L-yFEWNwXW00" /> </object>

Motivation

The architecture of Erpione is heavily inspired by the Screenplay Pattern, which fosters maintainability, extensibility, and a strict separation of concerns. By structuring the system around clear roles and responsibilities — such as test data generation, API handling, and integration with the E-Rezept Fachdienst — Erpione enables users to focus on their specific development and testing needs without being burdened by the complexity of the entire E-Rezept ecosystem. The Screenplay Pattern's emphasis on modularity and composability supports rapid adaptation to changes in specifications, facilitates the addition of new simulation features, and ensures that each component can be tested and evolved independently. Ultimately, this architectural approach helps Erpione fulfill its goal of providing a robust, user-friendly simulation environment for end-to-end testing and development.

Contained Building Blocks

Name	Responsibility
RestController	Exposes REST endpoints for client systems and the frontend; routes requests to the appropriate simulation or integration logic.
HermesService	Translates between the Hermes Data Model and the FHIR-based data model; see Hermes.
ErpioneActorService	Manages the actor stage and helps controllers select actors; see Actor Model. This concept is heavily inspired by the Screenplay Pattern.
ErpioneActor	Models an actor in the Screenplay Pattern, encapsulating the state and behaviour of a test actor; see Actor Model.

6. Runtime View

Erpione's runtime view illustrates how its main components interact during typical and critical scenarios. This section provides representative examples of end-to-end workflows, such as creation and dispensation of prescriptions. The focus is on the dynamic behaviour and communication between Erpione's building blocks, supporting stakeholders in understanding operational and integration aspects without requiring deep knowledge of the static architecture.

6.1. Scope and Purpose

The runtime view focuses on the dynamic behaviour of Erpione during representative workflows.

- What happens inside Erpione during a user-triggered action
- How the REST API, frontend, authentication, and internal services collaborate
- How Erpione communicates with the E-Rezept Fachdienst and related external systems
- How runtime behaviour differs between full-stack and simplified deployments

The participants of the scenarios are the external systems introduced in Context and Scope — above all the E-Rezept Fachdienst and the complementary systems that take over a prescription during its lifecycle — and the building blocks described in Building Block View.

The prescription lifecycle itself (draft, ready, in-progress, completed, cancelled) is owned and enforced by the E-Rezept Fachdienst and is specified in the api-erp documentation. Erpione does not re-implement this state machine; it triggers the individual transitions on behalf of an actor. The Typical Runtime Flows below show which REST endpoint of Erpione maps to which Fachdienst operation along this lifecycle.

6.2. Recurring Runtime Concepts

The workflows described below are all composed of the same few cross-cutting concepts: actor resolution, Hermes translation, execution as an actor and local persistence. They are described once in Cross-cutting Concepts; the individual flows refer to them instead of repeating the details.

6.3. Typical Runtime Flows

All domain workflows in Erpione follow the same basic shape, regardless of whether they are triggered by the Web UI or by an automated client. A request arrives at the REST API together with an `x-actor` header; the controller resolves the actor from the actor stage and translates the payload from the Hermes Data Model (or takes a raw FHIR resource) into the FHIR resource required for the operation. The operation is then executed as the selected actor, which signs, authenticates and sends one or more FHIR operations to the E-Rezept Fachdienst. The response is translated back into the Hermes Data Model, stored locally and returned to the caller as JSON.

This means that Erpione itself does not implement the prescription lifecycle; the authoritative state of a prescription is always held by the Fachdienst. Erpione's

responsibility at runtime is to orchestrate the correct sequence of FHIR operations for a given actor role, hide the complexity of signing, authentication and FHIR profiling, and keep a local copy of the results so that clients and the Web UI can inspect them later.

The following subsections show this pattern for the most relevant scenarios: creating a prescription as a physician, taking over and dispensing a prescription as a pharmacy, and the two ways of reaching the REST API — interactively through the Web UI in a full-stack deployment and programmatically in a simplified REST-only deployment.

6.3.1. Prescription creation

A physician actor creates a pharmaceutical prescription with `POST /api/erp/pharmaceutical`. The request carries the `x-actor` header and an optional body; the `direct` query parameter switches to a direct assignment ("Direktzuweisung").

1. The controller resolves the actor from the actor stage. Only actors with an HBA (physicians) are accepted here.
2. The body is translated into a KBV prescription bundle:
   - `Content-Type: application/json` — the Hermes prescription payload is converted into FHIR. Omitted fields are faked so the resulting bundle still conforms to the active KBV profiles; an empty body therefore produces a complete random prescription.
   - `Content-Type: application/fhir+xml` — the body is taken as a FHIR template. An optional `kvnr` query parameter replaces the patient's KVNR in the template.
3. The controller selects the prescribe operation for the bundle and executes it as the resolved actor. The flow type (160, 169, 200, 209 or 166 for T-Rezept) is derived from the insurance kind in the bundle, the `direct` flag and the medication category.
4. Executed as the actor, the operation performs two Fachdienst calls:
   a. `Task/$create` with the flow type — the Fachdienst returns a draft `Task` containing the prescription ID and access code.
   b. The KBV bundle is adjusted (prescription ID, current dates, MVO dates) and signed as QES with the actor's HBA through the software Konnektor. `Task/$activate` uploads the signed bundle; the Fachdienst validates it and returns the activated `Task` (status `ready`).
5. The activated task is translated back into the Hermes Data Model, enriched with the prescription data and the signature information, stored locally and returned as JSON. The response contains everything a pharmacy needs to claim the prescription (prescription ID and access code); a Data-Matrix-Code can be fetched afterwards with `GET /api/erp/dmc/{pid}`.

DiGA prescriptions (`POST /api/erp/evdga`, flow type 162) and the KTR use cases (`/approve`, `/decline`) as well as the EU pharmacy workflows under `/api/eu` follow exactly the same pattern with different operations towards the Fachdienst and are therefore not described separately.

6.3.2. Prescription claim, dispensation and close

A pharmacy actor takes over a prescription in up to three steps. All endpoints are located under `/api/erp/{pid}` and resolve the actor as an institution, i.e. an SMC-B is sufficient.

1. **Claim** — `POST /api/erp/{pid}/claim?ac=<accessCode>` calls `Task/$accept` at the Fachdienst. The returned accept bundle contains the `Task` (now `in-progress` with a `secret`) and the signed KBV bundle. Erpione decodes the KBV bundle, translates both into the Hermes Data Model, stores them (overriding any already known data for this prescription) and returns the task including the secret.
2. **Dispense** (optional) — `POST /api/erp/{pid}/dispense` calls `Task/$dispense`. The body is either Hermes JSON or FHIR XML describing the dispensed medication and is translated into the dispense parameters; if omitted, matching dispense data is generated. The Fachdienst keeps the task `in-progress` and Erpione returns an empty acknowledgement.
3. **Close** — `POST /api/erp/{pid}/close` calls `Task/$close` with the close parameters (same input options as dispense). The Fachdienst completes the task and returns the signed receipt, which is translated into the Hermes Data Model, stored and returned. The raw FHIR receipt bundle can be re-fetched later with `GET /api/erp/receipts/{pid}`.

If a pharmacy cannot fulfil the prescription it can hand it back with `POST /api/erp/{pid}/return` (`Task/$reject`), and any involved institution can delete it with `DELETE /api/erp/{pid}/abort` (`Task/$abort`).

6.3.3. Manual exploratory workflow via Web UI

In the full-stack deployment a tester works with Erpione through the browser. The Web UI is a static single-page application served by its own Nginx container and reached through the reverse proxy under `/ui`. It never talks to the Erpione service directly; all calls go to `/api/**` on the reverse proxy, which forwards them to the OAuth2 BFF.

Whether the user has to authenticate depends on the deployment configuration:

- **Authentication enabled** (default): the Erpione service runs as a stateless OAuth2 resource server and requires a valid JWT for every `/api/**` request. The BFF therefore redirects an unauthenticated browser to Keycloak (`/oauth2/authorization/…`, `/login`), performs the OIDC authorization code flow, keeps the tokens in a server-side session and hands only a session cookie to the browser. For every subsequent `/api` call the BFF attaches the access token as `Authorization: Bearer` header before relaying the request to the Erpione service (token relay) and refreshes the token transparently when needed.
- **Authentication disabled**: when the Spring profile `noauth` is active, the Erpione service permits all requests. This mode is intended for local or isolated test environments where Keycloak is not needed.

Once a request reaches the Erpione service, the processing is identical to the API workflow described above. The Web UI only provides a convenient way to select an actor (`GET /api/actors`), fill in Hermes payloads, trigger the prescribe/claim/dispense/close actions and inspect the persisted results (`GET`

`/api/erp/{pid}`, `GET /api/erp/prescriptions/{kvnr}`, `GET /api/erp/receipts`). Actor resolution, Hermes translation and persistence are handled locally inside Erpione; only the FHIR operations themselves leave the deployment towards the Fachdienst via the SZZP.

6.3.4. Automated API workflow

An external test system drives Erpione purely via REST. In the simplified deployment the Erpione service exposes its REST API directly, typically with the `noauth` profile; in the full-stack deployment the client has to obtain a JWT from Keycloak and send it as bearer token (or go through the BFF like the Web UI does).

The request/response lifecycle for a single call is:

1. The client sends an HTTP request to `/api/erp/…` with the `x-actor` header and, where applicable, the prescription ID in the path and `ac` / `secret` as query parameters.
2. The controller resolves the actor, translates the body and executes the operation as the actor as described in the two prescription flows above.
3. The response is always JSON in the Hermes Data Model (the task, the receipt or an empty acknowledgement) or an `ErrorData` payload in case of failure (see Error Handling). The only exception is `GET /api/erp/receipts/{pid}`, which returns the raw FHIR receipt bundle as `application/fhir+xml`.

The API is JSON-first: the Hermes Data Model is the primary and stable contract, while raw FHIR is accepted as *input* for selected endpoints (see Translation). In both cases the FHIR resources sent to the Fachdienst are built by Erpione; the client never communicates with the Fachdienst directly.

The results of these calls are stored locally so that the lookup endpoints and the Web UI can inspect them later; this local copy is a convenience cache and never the authoritative state of the prescription.

6.4. Sequence Diagrams

The first diagram shows the full-stack deployment: a tester prescribes through the Web UI, the request passes the reverse proxy and the BFF (including the Keycloak login when authentication is enabled) and finally reaches the Erpione service, which performs `$create` and `$activate` at the Fachdienst.

The second diagram shows the simplified REST-only deployment from the

perspective of an automated client acting as a pharmacy.

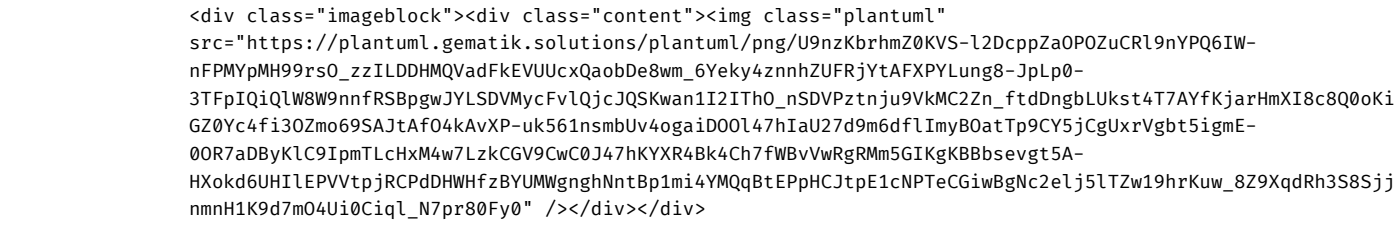

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9orLTzF6a4GVh-
_Oc9zW0GEQfE6454HeMgr67yqVV1bsHjWmztkTNTFfSR_lNExnm5MiQOf3t3iVD-
tCz_CmAwnynZt0g0NurPfU7JF065txIl08UEpYLQPZ7egfiW7nDQdpysTyHAXT8HwBLh4deA5wet1ZQAjrfYzBkgZfUYAT6QGCuEbT2xy
PLLuC1Yq-
gqYUYOa5kUzZwr1irb8xJOAFmwCPTfcIHomp2gP9IFKWPa9cJBD4Z3uCqF9yHoDOHFinc8YWQEqX0e8OWKNAPCMjdgne6EucXkBoQsiO3
W9mQAna0idp0ebgrl0372734Rg0RRwEgLpX2lKTuBZriBEPkxBclubIVMKj5h5CMelP7Ny75O8kfwMeeIbCYVu77oCEa6p2fkwa-Uu6--
vGsHyqvLKU3TLTeeprBvQDspFAHUeZI2ZP75jD0fGHa2Mh9lQhr_YBqpjhHmmFet8DsCTOHm5GQS3lPY912hRZ7D2LQ7J2RoxKDzpza0R
BePNrz1WgMYWJXkFgOYU6ZpdxJFU2SFmLZxKNP9js4sdJ2UCp_S2HoOHvnkfJD2UqEN2PdaZX_fwgjLznV5EechWOVNbhSV8OZlbJECAh
LJDE45SyXSQ3TSYpSFXlR3J2YLO8zUgjp9CKO-LJhh6NrGDY4TilxzjqFbm6bkvPcRMM7WG-
2Z1nb5PlKCyvfKNiABsW33r5SMk2nxICiK8JhS6XaOU8_Xw-1r6Bh18myyAiQmwDKfMQ0xwdcBfdVAFaAi8_nGe-D68k8RTOSdqoaiJ5-
U5Rtd0TxSDEyrc6ORAwTNmMms4v758AyiraW_vV1m9azBCK2tAESpYlv-HgE2Myr43lo-GtmRar-3uv19-d9-
LRLrnP30LcbPMPPeZDC1RiTx_HN8NVkYGHeIp-7rDNoJvPjiNm7rlpqlDFyU84iXjAcqh-
lMA_KbgvqVCic8LLUxpDiy_txE9wZzQx9Bw5tzzDM-Pwr0RdBr4ZhJYKF6hi6c12kWmipLAnJrNNzoN9R2wOJsMAByarU20VcZfx-
utv8YS7m00" />

7. Deployment View

This section describes the technical infrastructure on which Erpione is executed.

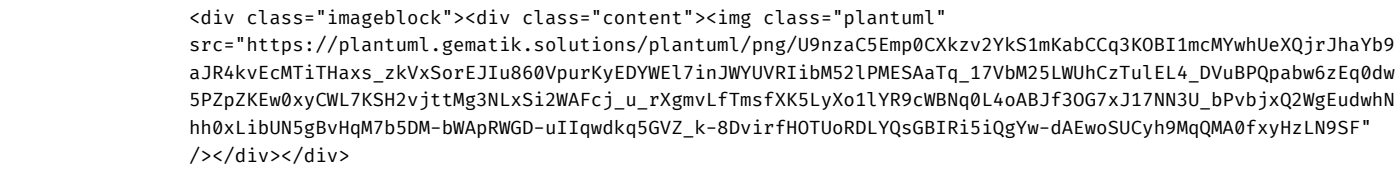
7.1. Infrastructure Level 1

Service Deployment



as a lightweight alternative the Erpione service can also be run without the Web UI and authentication server, directly exposing the REST API.

Simple Service Deployment



Motivation

Erpione is designed for flexible, secure, and reproducible deployment. By running all core components (the Erpione service, authentication server, proxy etc.) as Docker containers, the system can be easily started, stopped, and managed in any environment that supports Docker.



An SZZP acts as a central VPN access point or proxy. It protects the central network (e.g. the telematics infrastructure in the healthcare sector) by routing all connections through it. This connectivity to the TI must be provided by the runtime for the Erpione to run properly. Make sure to read and understand the implications as described in TI Connectivity

Quality and/or Performance Features

- **Isolation:** Each component runs in its own container, providing process and resource isolation.
- **Portability:** The entire stack can be deployed on any host with Docker support.
- **Configurability:** Environment variables and Docker Compose files allow for easy configuration of endpoints, secrets, and runtime options.

Mapping of Building Blocks to Infrastructure

- **SZZP**: Handles all incoming and outgoing traffic to the TI.

7.2. Infrastructure Level 2

7.2.1. Resource Server Container

- Runs the Spring Boot application with all business logic, REST API, and simulation features.
- Connects to the database and Keycloak for persistence and authentication.

7.2.2. Authentication Server Container

- Keycloak provides authentication and authorization services for Erpione.

7.2.3. Reverse Proxy Container

- Nginx routes incoming requests to the appropriate backend services (e.g., REST API, Keycloak) and handles TLS termination if needed.

7.2.4. Web UI Container

- Nginx serves the static files of the web-based user interface, allowing users to interact with Erpione through a browser.

8. Cross-cutting Concepts

This section describes the concepts that span several building blocks of Erpione and recur in every runtime scenario. They are introduced once here so that the Building Block View and the Runtime View can refer to them instead of repeating the details: how clients exchange data with Erpione without having to deal with FHIR (Hermes), how actions are performed on behalf of a test actor (actor model), how results are kept locally (persistence), how failures are reported (error handling) and which connectivity Erpione needs towards the Telematikinfrastruktur.

8.1. Hermes – FHIR Abstraction

Hermes is Erpione's FHIR abstraction: a domain-level data model together with the translation layer that maps it to and from FHIR. The name refers to the Greek messenger god who mediates between two worlds – here between the business view of a prescription and the FHIR representation required by the E-Rezept Fachdienst. Hermes consists of two parts:

- The **Hermes Data Model** is the simplified representation of the E-Rezept domain data exposed by Erpione's REST API as JSON. It describes prescriptions, dispenses, receipts and related business objects without any FHIR-specific semantics.
- The **HermesService** is the translator that maps the Hermes Data Model to the FHIR resources expected by the E-Rezept Fachdienst, and FHIR responses back into the Hermes Data Model.

8.1.1. Hermes Data Model

The Hermes Data Model is implemented as plain Kotlin/Java classes which are shared across the REST API and serialised as JSON. The classes are designed to be simple and intuitive: they carry the business data a tester cares about – patient, prescriber, medication, dispense information – and leave out the structural and terminological ceremony of FHIR (bundles, references, profiles, codings). At the same time they support all data necessary for realistic testing scenarios, so that nothing relevant for the E-Rezept Fachdienst is lost in translation.

The main advantage of this decoupling is stability. The FHIR profiles of the E-Rezept are more likely to change over time than the business view of a prescription. A profile upgrade is therefore handled entirely inside the HermesService; the Hermes Data Model can stay stable even when the underlying FHIR profile version changes, and in many cases it does not need to change at all. The Hermes Data Model is the primary and stable contract of Erpione's REST API.

8.1.2. Translation

Clients talk to Erpione in the Hermes Data Model, not in FHIR. Before an operation is executed, the HermesService translates the incoming payload into the FHIR resource the Fachdienst expects (for example a KBV prescription bundle or the parameters of a dispense). After the operation, the FHIR response of the Fachdienst is translated back into the Hermes Data Model before it is returned to the client. In both directions the FHIR resources that reach the Fachdienst are

built by Erpione; the client never communicates with the Fachdienst directly.

FHIR stays optional rather than hidden. For selected endpoints (`/pharmaceutical`, `/evdga`, `/dispense`, `/close`) a client may alternatively send a raw FHIR resource with `Content-Type: application/fhir+xml`. Such a resource is taken as a template and used as-is, so that vendors can test their own FHIR resources against the Fachdienst without re-modelling them in the Hermes Data Model. Raw FHIR output is available where it is relevant for testing, e.g. the signed receipt bundle via `GET /api/erp/receipts/{pid}`.

8.1.3. Faker Capability

A second advantage of Hermes is its faker capability. When a data field is not relevant for a scenario, it can simply be omitted. Hermes fills the gap with a syntactically fitting, profile-conformant value, so that the resulting FHIR resource is still valid for the active profiles. Fields that are provided by the client are never overwritten. As a consequence an empty request body yields a complete, valid resource – for example a complete random prescription – which makes it possible to generate synthetic FHIR data sets that are accepted and processed by the E-Rezept Fachdienst.



The REST API provides dedicated `/faker` endpoints which you can use to play around with Hermes and inspect the generated data.

8.2. Actor Model

Erpione's architecture is heavily inspired by the Screenplay Pattern: every business operation is performed *by* an actor. An `ErpioneActor` encapsulates the state and behaviour of a test actor – its identity (HBA or SMC-B), its role and everything needed to talk to the Telematikinfrastruktur. The `ErpioneActorService` manages the stage of actors and gives controllers a simple way to select and work with actors during a workflow. This keeps actor handling separate from request processing and domain logic.

8.2.1. Actor Roles

These are the main actor roles in the context of Erpione:

- **Arzt**: Represents a physician who uses the PVS of its LEI.
- **Apotheker**: Represents a pharmacist who uses the AVS of its LEI.
- **Kostenträger**: Represents a health insurance that approves DiGA prescriptions.
- **EU-Apotheker**: Represents a pharmacist from an EU country who can receive and dispense prescriptions via the NCPeH.


```
<div class="imageblock"><div class="content"><img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9nza4rEmp0GXVS-
XINs8O1KMP4LjLHD5x3j0InxsaH9xMWy4Ig8st0JBaRYc3PUW5VsUy_V_4XFbmvAniUZD3da7-
7EURP9dfm_AzsUq0tMR5qtEdS0uccLFnvl2OS6C76Zzyb-gGSF_qA5EAg_eRDxXIxGWmUjF5pH4tY_15TL9NOYkg-DXQXl7qILvn4bx-
C-rfuKqd3k6AiR0zmJ83YwzC1OCSrGSEMdeiYbdDvboOeDlbFGZGkxapBiC5YzenhQ-A-
l5PwLRZmjCozkh689lxzECTTI0jaTUCgnkeTwaM2XeSouDvkLivjA8p0LRfsa-Zm1MuCrun9-0BR0f540" /></div></div>
```


The use cases of `Arzt` are:

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9oDaarEmp0GXVS-
XIKsxQAbW8IYA8fQIjCD2uHWNLdsfB5musWy4MeHr-82N0pRf7yBscPb-xttwSqecQErWPnn_mb95lbdF7C-
d1nTENyGyds9jgtLr1glN04aT_UZstBliAW0ZzHE-s9Rwfk3az0aASN_q3zr1pr0jmwaSB131_3y45mKHJ9BElL9rz2zJs-
IOZJgKDbrju-n8u7KhWnZbLR07G62fNK3p419d2n7lQo8AuqWITkQiJJCbdBXGXdaMHRkUSxI2MuelYiRTvfbSRTUYiQXiX_rCD-
gAMy0Iuihds2CzPuhWIknWPQqanKuFXkym0OQug1htyiOgFk1tclbGY88WaKJCp9af31zrdlKyyabGQMNmjjVppi9eyERTrZpynqQxgqA
313q6Uj66uH91_c9fllftObGR7cPTLlpZBjhUkXYOwYL_mD-0LdY_sC0" />

The use cases of `Apotheker` are:

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9oDairEmp0GXEz-
2ajSog6bW8IYgefQ4EM2a43YNBdsfh58x6Yz1W7YjJXnwujXjm7SFzISuimtizb9XevUAyWP3vUGP96_B-yvxutNZfnV2laqH-
kDkh9LK8u0ilEB_bdnHrXKW6jggtsmdzH71_-6PbaXzeUkrD_G9DexaCB1RtGClac39vD9Tfsrwgqsq3w_Eiqc_NuRDJnf-nWv4aY-
hXWhjGBk210ilJR0AYY8a-MevoLnfH4aQMiO6yJT1bou-3B8XyDupdCs63UMIdXgnQOKM0kf7GLaMN9WbewUiY-
cbwV4W3U0XSKwE1dhN7SVu0qQujNYqyaIp75Cx7WvLL01mN62Y1dw8Y5a9NIT0cyU5r-
XVfWb509vD2ccj6l0E1qAICEiYzFklIRoPiu13VZjeQ4Sq0q5yax84hKi2SmEIgYfj7NjZPOYjhbXU0PqO5HYYSTD2sD9SzrOPZTFLlyh
qws1ATcklmzToumD79BPVfh3ng_gEepTAkqGsuxYKjieoE6t_mOZZbQw" />

The use cases of `Kostenträger` are:

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9nzaS5Ema0GXk_x59DuWKCHDJ4DQHeGAGSzcNWsstPQDYoxpUnM
0iQtyJ6yyMBkrW958Zrrz__wvPzfPEn6Oip0FJopck2ZUGSOJ4wE00yyMvQaQvLFjNJ95M9uTp-
yBOw4fXpf96spJxQtlXgyA0t3Wlyl_KqFqewwDfXnWmUr5y-
xuYH9mbdOfiz2ONi_lGcJuR1LHTVjFiR6ShBrIZAs43c2iKYexQP29h6mO3MGA1SMSa6OMQ4LOoC_smYuyMK8eiYVuvYDdhJxN5dQVPKj
iBG-RpRiW0OVv7gj1d4JZQ12AZIj7CjOxr7C9p0BNd2BbGMUKbsWwdjFhyxUCid5gl-NsjQq-
tQxnRI34jgQr1cRubf8ASeEoQkAz3kUeJoLk50TCCTCkcNs6VF5Wy1F4SITOiTXp3dI5hh8xAjSXFOrkj0OLUx-u0ymUUFX"
/>

The use cases of `EU Apotheker` are:

<img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9o5aKrEmp0GXVS-
nKXisaLB0Gb5LHMrj0aRLaXTLwuzIQmcjZLsG0LnBIx0nOXBI5EL7wzilpUVvurdpkzBZ1aqYmjl25uFUu3nukG8SCV5BYTJQxaqPQDS8
4OtjvFhxEWm995Er5PxOz_KjSC_eL6KyT-XNse7xQ5hXu8xxD01VDy7fsaQ9L6hFYYDxVtoAaedanOrkspdCNUUaw-
hah52IGJdaT3ulKLMOkR16o2L5nwa8XHU6StOD6IR0dUX6OBPB9pZc4sJDIoiyGNkMbqBYqLZE8ouSHpyOscUzJXknIbOfCnGrVWP6wom
CZbnMoWd2eGLzwY1RwdEK0y3Sb2BZUnCmWtP856w_7XtMqu8oUWHNz1wF2l2UcC9dI1bGunGkDZcV8lmPFInJeEg80IfNScrx9TBvIngX
qF6s5Vpen1b59ytTOpbVh2UjVAllNkwxvoZbitFVG9YF-MH" />

8.2.2. Actor Resolution

Every request to the REST API names the actor who shall perform the action via the `x-actor` header. The controller looks this actor up on the actor stage and checks that the actor's role is suitable for the requested operation: creating a prescription requires a healthcare professional with an HBA (a physician), whereas pharmacy operations only require an institution with an SMC-B. An unknown or unsuitable actor is rejected before anything is sent to the Fachdienst (see Error Handling).

8.2.3. Execution as an Actor

The business operation itself is modelled as a use case that is performed *by* the resolved actor. Executing a use case as an actor means that the actor signs documents with its HBA or SMC-B through a software Konnektor, authenticates against the IdP and sends one or more FHIR operations to the E-Rezept Fachdienst on its own behalf. Controllers only decide *which* use case to run for *whom*; the actor encapsulates everything needed to actually talk to the Telematikinfrastruktur.

8.3. Local Persistence

The result of each operation is stored locally per prescription ID: the current task state together with access code, secret, prescription data and signature information after a prescribe or claim, and the receipt after a close. These stored copies feed the lookup endpoints (`GET /api/erp/{pid}`, `GET /api/erp/prescriptions/{kvnr}`, `GET /api/erp/receipts`) and the Web UI. They are a convenience cache only — the authoritative state of a prescription always lives in the Fachdienst — and expired prescriptions are purged periodically.

8.4. Error Handling

When an error occurs, Erpione represents it with `ErrorData`. The error payload contains an `ErrorType` that indicates whether the error originates in Erpione itself or in the Fachdienst. A human-readable message explains the problem, and optional issues can be included to narrow down the root cause.

So for example, if a prescription creation fails due to an invalid role of the actor, the `ErrorType` would indicate that the error is produced by the Fachdienst, and the issues would explain the root cause. In the following case for example we tried to create a prescription with an actor that has the wrong ProfessionOID, which is rejected by the Fachdienst:

```
{
    "message": "Actor Telesphorus expected a response of type 'ErxTask' but received
'OperationOutcome' instead",
    "type": "FACHDIENST",
    "issues": [
        {
            "severity": "Error",
            "details": "The QES-Certificate does not have expected ProfessionOID."
        }
    ]
}
```

Or if try to claim a prescription which is unknown to the Fachdienst, the Fachdienst would reject the request with an `OperationOutcome` and Erpione would return the following error payload to the client system:

```
{
  "message": "Actor Hygieia expected a response of type 'ErxAcceptBundle' but received
'OperationOutcome' instead",
  "type": "FACHDIENST",
  "issues": [
    {
      "severity": "Error",
      "details": "Task not found for prescription id"
    }
  ]
}
```

While on the other hand, if Erpione itself fails to process a request or runs into an internal error, the `ErrorType` would indicate that the error is produced by Erpione, and the issues would explain the root cause. In the following case for example when we try to call a REST endpoint that is not implemented, Erpione would return the following error payload to the client system:

```
{
  "message": "No static resource api/erp/claim/160.000.114.452.028.14.",
  "type": "ERPIONE",
  "issues": [
    {
      "severity": "ERROR",
      "details": "No controller found for: POST api/erp/claim/160.000.114.452.028.14"
    }
  ]
}
```


Or if we try to perform an action with an actor that is not present in the stage of actors or not allowed for this operation, Erpione would return the following error payload to the client system:


```
{
  "message": "No such actor with id 'Erpione'",
  "type": "ERPIONE",
  "issues": [
    {
      "severity": "ERROR",
      "details": "ActorNotFoundError: No such actor with id 'Erpione'"
    }
  ]
}
```


The following diagram illustrates the two error categories. In the first case the Fachdienst rejects the request with an `OperationOutcome`; the actor raises an `ErpResponseExpectationError`, which the central `ErpioneErrorHandler` (`@ControllerAdvice`) translates into `ErrorData` of type `FACHDIENST`, preserving the Fachdienst status code and issues. In the second case Erpione itself fails before contacting the Fachdienst (here: unknown actor), and the error handler returns `ErrorData` of type `ERPIONE`.


```
<div class="imageblock"><div class="content"><img class="plantuml"
src="https://plantuml.gematik.solutions/plantuml/png/U9obLS5o6ZaGlSzNT7dt05LcOBEfBSfbl2O4rbHbZSjsJY47MTE0
bXbf8lN49Yx_-xQaOO0avX9EGdhzkllra-RSqIR7YmJu9oGP2yzX3P0E3_u2l1DolRIcqjd8v7po6sB_pxzwRnOxXB4PseFJ-kmbsR9-
T7YKjDzVYDT9usb3kaTTEPJ2OKFjYV_P9vvC9lrnlpxze3JM-wC_-
fDUhwOwxzPwN3eIbgeYJqH5HbV50zh4hPKkXHK5EFnIePRuBpedbZZCrLA3H4sCIXX2IgfIQ8AJKQvu6-
usZh2OwnQconG87O4dBWGfezid81p8W4mUp1ESZ6t9-mXtQBygYITRENjUbmF-MwOQ6KtMv3dQI4Gs_nvM4uQPDgICvJQN-
3tuOzGo02fvZ8eR6fQ6LhX66xi89debP6ojiLT2PrmfHrxkwXvclgB8i8g017K6BCQXdEFEBNx3akPw8kGgOyKSXQX5bYI30UnsmU9_AC
a1LyThdXVl3GP9b1aw5q4dE8ERsTqzT4MfkcZBxdEfifUkp8Kg_XPoaARfN3zrGbTdS5wkX2s4t5ma8PX9We3CiaIQSbP5bP_epDwlh7b
iXRXfrauYhBFBUeKYfvKK5lV26jPACcJaYxYnwAHLfJzE7nMjMbpWASlSdkkqHBimjXYwcAYTS1n7nujyjkswTI_Sklkxa9A5QoVnb66B
hEa-89vzvpMixZrXJ6UFSSOPWsLd5KbJ84G_-t3GXc3XBroJOji8UIs0JS1cBOrsE7ugUJI1A7Z2tukmXHaCG_vtbJU1xvCgaV-
GDtQwK_C8TwDfxIj6nzMH6AwcdMpn1w65A7ye9bNELVZfyroJWQbVlnSaeENmArf5crF8a8JA7I-
KM6h3yvPkBu7l8xgIBRRdO3ODLNoVJOONyrXPO7wcJObdC1cEhjvFnzTtzwTGn8Ve548jxjFdbt0Tjk-8yY-
JPY7Ha_poDQZqMfj77MtnwtUXyOUlzThGnBle-
F32wzD0dxnbEFZrkJrKAi_sXhEmfW3Snh7_k5NdG8i2zyIKWZyhhmpgRUyjS04GAlZ9TCQtDzFPzVYdexdajus_BVy3b9s4C000"
/></div></div>
```


8.5. TI Connectivity

Erpione is designed to communicate with the E-Rezept Fachdienst and therefore needs TI connectivity. The `api-erp` documentation provides one detailed way to set this up in TI-Konfiguration, but other connectivity solutions may also be possible depending on the deployment environment.

Independent of the chosen route, the Fachdienst identifies the calling system via the `User-Agent` header. Patient actors (simulating the FdV) additionally

communicate via the Internet route and require an `X-api-key`. These credentials are not shipped with Erpione; operators have to provide their own values as described in E-Rezept Fachdienst Credentials (`application-secrets.yaml`).

9. Build, Release & Operations

This section covers the build process, deployment procedures, and operational aspects necessary to set up and maintain Erpione in your own infrastructure.

9.1. Overview

Erpione is a distributed system composed of multiple modules written mainly in Java and Kotlin. The build and deployment process involves:

- Building multiple Maven modules (Java/Kotlin backend components)
- Creating Docker container images for each service
- Orchestrating containers via Docker Compose
- Managing secrets and configuration

9.2. Prerequisites

9.2.1. System Requirements

Component	Requirement
Docker	Minimum version: 29.5.3 (tested and recommended). Older versions may work but without guarantees. Docker Engine with Docker Compose (v2.0+). See Docker Installation Guide
Memory	**Without authentication (no Keycloak/BFF):** Minimum 1GB+ **With full stack (Keycloak + BFF):** Minimum 2.5GB, Recommended 3GB+ Total stack consumption is approximately 2.5GB with authentication enabled. Keycloak and BFF are the primary memory consumers. Allocate 3GB+ for comfortable operation and headroom.
Network	Connectivity to TI (telematics infrastructure) environments Proxy/SZZP access for production deployments (see Deployment View)

9.2.2. Build Tools

Tool	Version	Usage
Java Development Kit (JDK)	21	Required for building and running all backend services (erpione-server, erpione-oauth2-bff)
Maven	3.9.0+ (bundled via `./mvnw`)	Used to build Java/Kotlin modules. The repository includes the wrapper script `./mvnw`.

Tool	Version	Usage
Git	Latest	For cloning the repository

9.2.3. Supported Operating Systems

The only hard requirement is **Java 21 capable JVM** and **Docker**. Erpione runs in containers, so the underlying OS is largely irrelevant.

- **Linux**: Any distribution capable of running Java 21 and Docker (Ubuntu, Debian, RHEL, CentOS, Fedora, Rocky Linux, AlmaLinux, etc.)
- **macOS**: Any version that supports Java 21 JVM (M1/M2 ARM64 and Intel architectures supported)
- **Windows**: Windows 10/11 with WSL2 backend (to provide Linux kernel and Docker support). Requires Java 21 JVM in WSL2 environment

9.3. Build Process

9.3.1. Repository Structure

```
erpione-application/
├── erpione-server/              # Main REST API service (Spring Boot, Java/Kotlin)
├── erpione-oauth2-bff/          # Backend for Frontend (Spring Boot, OAuth2 client)
├── erpione-frontend/            # Web UI (static HTML + JavaScript, served by Nginx)
├── erpione-rest-data-ts/        # TypeScript data models
├── erpione-cli/                 # Command-line interface client (Java/Kotlin)
├── erpione-rest-client/         # REST client library (Kotlin)
├── erpione-keycloak/            # Keycloak authentication server (container)
├── erpione-reverse-proxy/       # Nginx reverse proxy configuration
├── erpione-integration-coverage/# Integration tests & coverage reporting
├── pom.xml                      # Parent Maven configuration
├── dc-erpione-local.yml         # Docker Compose for local development
└── dc-erpione.yml               # Docker Compose for production
```

9.3.2. Building Java/Kotlin Modules

Full Build (all modules)

```
cd $(git rev-parse --show-toplevel)

# Clean and build all modules, skip tests
./mvnw clean install -DskipTests

# Package the main erpione-server JAR
./mvnw package spring-boot:repackage -f erpione-server/pom.xml -DskipTests
```

Build Individual Modules

```
# Build only the server
./mvnw clean package -f erpione-server/pom.xml -DskipTests

# Build only the BFF
./mvnw clean package -f erpione-oauth2-bff/pom.xml -DskipTests
```

```
# Build only the CLI
./mvnw clean package -f erpione-cli/pom.xml -DskipTests
```

Build Artifacts

After successful build, look for:

Component	Artifact Location
erpione-server	`erpione-server/target/erpione-server-*.jar`
erpione-oauth2-bff	`erpione-oauth2-bff/target/oauth2-bff-*.jar`

9.3.3. Building Frontend

The frontend consists of static HTML and JavaScript files served directly by Nginx. No framework (Vue, React, etc.) or build compilation is required.

Frontend Structure

The `erpione-frontend/` directory contains:

- Static HTML files
- JavaScript files
- CSS stylesheets
- Images and assets
- Nginx configuration (`default.conf`, `entrypoint.sh`)

Frontend Deployment

The frontend files in `erpione-frontend/dist/` are copied directly into the Nginx container during image build:

```
FROM nginx:alpine
COPY dist/ /usr/share/nginx/html/
```

No build step is required for the frontend. The static files are served as-is.

To update the frontend:

1. Modify HTML, CSS, or JavaScript files in `erpione-frontend/`
2. Rebuild the Nginx container: `docker compose -f dc-erpione-local.yml build ui`
3. Or copy updated files directly to running container: `docker cp ./erpione-frontend/dist/. erpione-ui-${ENVIRONMENT}:/usr/share/nginx/html/`

9.3.4. Building Docker Images

Build All Images

```
cd $(git rev-parse --show-toplevel)

export ENVIRONMENT=rudev
export VERSION=$(git describe --tags --always)
export COMMIT_HASH=$(git rev-parse HEAD)
export BUILD_DATE=$(date -u +'%Y-%m-%dT%H:%M:%SZ')

# Build backend JARs
./mvnw clean package -DskipTests -f erpione-server/pom.xml
./mvnw clean package -DskipTests -f erpione-oauth2-bff/pom.xml

# Build images using docker compose
docker compose -f dc-erpione.yml build
```

Docker Image Specifications

Service	Base Image	Size	Purpose
erpione-server	bellsoft/liberica-openjdk-alpine:21	~1GB	Main REST API backend
erpione-oauth2-bff	eclipse-temurin:21-jre-alpine	~500MB	OAuth2 session management frontend
erpione-frontend	nginx:alpine	~100MB	Web UI serving static assets
erpione-reverse-proxy	nginx:alpine	~100MB	Request routing and optionally TLS termination
erpione-keycloak	quay.io/keycloak/keycloak:latest	~800MB	Authentication and authorization server

Build Arguments

Each Dockerfile accepts build arguments for metadata:

```
--build-arg VERSION=1.0.0
--build-arg COMMIT_HASH=abc123def456
--build-arg BUILD_DATE=2026-08-19T09:00:00Z
--build-arg UID=1000
--build-arg GID=1000
```

9.4. Configuration & Environment Variables

9.4.1. Environment Files

Create a `.env` file in the repository root for Docker Compose:

```
# Core Configuration
ENVIRONMENT=rudev
```

```
ERP_FHIR_PROFILE=1.6.0
ACTIVE_SPRING_PROFILES=rudev,noauth
PUBLIC_PORT=8080

# OAuth2 / Keycloak
OAUTH2_ISSUER_URI=http://keycloak:8080/realms/erpione
OAUTH2_CLIENT_ID=bff-client
OAUTH2_CLIENT_SECRET=change-me   # TODO: Document secure secret generation
KC_ADMIN_PASSWORD=admin          # TODO: Provide secure default
KC_DB_USERNAME=erpione
KC_DB_PASSWORD=erpione           # TODO: Provide secure default

# Network / Proxy Configuration
PROXY_HOST=localhost             # TODO: Document SZZP proxy configuration
PROXY_PORT=3128
NON_PROXY_HOST=localhost,127.*,keycloak,redis,resource-server

# URLs
ISSUER_HOSTNAME=localhost        # TODO: Document production hostname setup
SPA_BASE_URI=http://localhost
ALLOWED_ORIGINS=http://localhost

# Redis
REDIS_PASSWORD=redis-secret      # TODO: Document secure generation
```

9.4.2. Environment Variables by Service

erpione-server (Resource Server)

Variable	Example	Description
ERP_FHIR_PROFILE	1.6.0	E-Rezept FHIR specification version to simulate
ACTIVE_SPRING_PROFILES	rudev,noauth	Comma-separated Spring Boot profiles. See Spring Boot Profiles
SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI	keycloak:8080/realms/erpione	Keycloak realm issuer URI for JWT validation
PROXY_HOST	proxy.example.com	HTTP proxy host for outbound E-Rezept backend connections
PROXY_PORT	3128	HTTP proxy port
NON_PROXY_HOST	localhost,127.*,keycloak,redis,resource-server	Hosts to exclude from proxy routing

erpione-oauth2-bff

Variable	Example	Description
SPRING_PROFILES_ACTIVE	docker	Spring profile (docker, local, prod, etc.)

Variable	Example	Description
OAUTH2_CLIENT_ID	bff-client	OAuth2 client ID registered in Keycloak
OAUTH2_CLIENT_SECRET	secret-xyz	OAuth2 client secret (must match Keycloak registration)
OAUTH2_ISSUER_URI	keycloak:8080/realms/erpione	Keycloak realm issuer URI
RESOURCE_SERVER_URI	resource-server:8080	Internal URI for backend resource server
SPA_BASE_URI	localhost	Public URI of the Single Page Application
ALLOWED_ORIGINS	localhost,https://example.com	CORS-allowed origins for API requests
REDIS_HOST	redis	Redis hostname for session storage
REDIS_PORT	6379	Redis port

Keycloak

Variable	Example	Description
KEYCLOAK_ADMIN	admin	Initial admin username
KEYCLOAK_ADMIN_PASSWORD	admin-password	Initial admin password (must be set)
KC_DB_USERNAME	erpione	Database username
KC_DB_PASSWORD	erpione-db-password	Database password (must be set)
KC_HTTP_RELATIVE_PATH	/kc	Relative path for Keycloak in proxy
KC_HOSTNAME	keycloak.example.com	Public hostname (for production)

9.4.3. Spring Boot Profiles

The `ACTIVE_SPRING_PROFILES` environment variable controls runtime behaviour:

Profile	Description
Environment Profiles (pick one)	
`rudev`	Running against the RU-DEV environment
`ru`	Running against the RU(-REF) environment

Profile	Description
`rudev`	Running against the TU environment
Feature Profiles (additive)	
`offline`	Run without connecting to the E-Rezept Fachdienst (offline mode)
`noauth`	Disable OAuth2/JWT authentication

Example combinations:

```
# Local development without authentication and without connecting to E-Rezept backend
rudev,noauth,offline
```

9.5. Secrets Management

TODO: This section needs to be populated. Questions for review:

1. **Secrets Generation**: How should operators generate secure secrets (Keycloak passwords, Redis passwords, OAuth2 secrets)?
2. **Secrets Storage**: Where/how are secrets stored in production? (Kubernetes Secrets, Docker secrets, environment files, HashiCorp Vault?)
3. **Secret Rotation**: What is the procedure for rotating secrets without downtime?
4. **Certificate Management**: How are TLS certificates generated and managed?

9.5.1. Development Secrets

For development (docker-compose), create a `.env` file with default values.

WARNING: Never commit `.env` files with real secrets to version control. Use `.gitignore`:

```
.env
.env.local
*.secret
```

9.5.2. E-Rezept Fachdienst Credentials (`application-secrets.yaml`)

Every request towards the E-Rezept Fachdienst must carry a `User-Agent` header that identifies the Primärsystem. Patient actors (simulating the FdV / E-Rezept app) use the Internet route and additionally require an `X-api-key` header. Both values are environment-specific credentials and are therefore **not** part of the public repository: the file `application-secrets.yaml` is excluded via `.githubignore`, and the public source tree does not contain it.

The server nevertheless expects these values. `application.yaml` includes the Spring profile `secrets` (`spring.profiles.include`), which makes Spring Boot load

`application-secrets.yaml` next to the other `application-*.yaml` files when present. Operators running their own Erpione instance must supply this file (or equivalent configuration) themselves.

The credentials are per environment profile (see Spring Boot Profiles) and live below `erpione.environment`:

Property	Required	Description
`erpione.environment.ti.userAgent`	yes	`User-Agent` sent on the TI route (split-DNS endpoints via SZZP/Konnektor). Format defined by gematik, e.g. `<ProductName>/<Version> <VendorID>`
`erpione.environment.internet.userAgent`	Patient actors only	`User-Agent` sent on the Internet route, which is used by patient actors (simulating the FdV / E-Rezept app)
`erpione.environment.internet.xapiKey`	Patient actors only	`X-api-key` for the Internet endpoints of the Fachdienst (`erp-*.app.ti-dienste.de`). Only required when patient actors (FdV simulation) are used; doctors and pharmacies communicate via the TI route exclusively.

Create `erpione-server/src/main/resources/application-secrets.yaml` from the following template and fill in your own values for every environment you intend to use. The `---` separated documents are activated per profile via `spring.config.activate.on-profile`:

```yaml
---
spring:
  config:
    activate:
      on-profile: tu
erpione:
  environment:
    ti:
      userAgent: "<your-user-agent>"
    internet:
      xapiKey: "<your-tu-x-api-key>"
      userAgent: "<your-user-agent>"

---
spring:
  config:
    activate:
      on-profile: ru
erpione:
  environment:
    ti:
      userAgent: "<your-user-agent>"
    internet:
      xapiKey: "<your-ru-x-api-key>"
      userAgent: "<your-user-agent>"

---
spring:
  config:
    activate:
      on-profile: rudev
```

```yaml
erpione:
  environment:
    ti:
      userAgent: "<your-user-agent>"
    internet:
      xapiKey: "<your-rudev-x-api-key>"
      userAgent: "<your-user-agent>"
```

Alternatively, the values can be provided without touching the source tree, e.g. as environment variables for the `erpione-server` container (`ERPIONE_ENVIRONMENT_INTERNET_XAPIKEY`, `ERPIONE_ENVIRONMENT_TI_USERAGENT`, `ERPIONE_ENVIRONMENT_INTERNET_USERAGENT`) or through an external config location (`SPRING_CONFIG_ADDITIONAL_LOCATION`). Standard Spring Boot property precedence applies.

Obtaining credentials

The `X-api-key` and the agreed `User-Agent` for the gematik test environments (TU, RU, RU-DEV) are issued on request by gematik. Please contact gematik Fachportal and state the environments you need access to.

WARNING: Treat `application-secrets.yaml` like any other secret: never commit it to a public repository and keep it out of Docker images that are shared outside your organisation.

9.5.3. Production Secrets

TODO: Document production secrets management strategy.

Recommendations:

- Use Kubernetes Secrets (if running on Kubernetes)
- Use Docker Secrets (if running on Docker Swarm)
- Use HashiCorp Vault for external secret management
- Use CI/CD platform secrets (GitHub Secrets, GitLab CI variables, etc.)

9.6. Deployment Procedures

9.6.1. Prerequisites Checklist

Before deploying, ensure:

☐ Docker and Docker Compose installed and running

☐ All required environment variables defined in `.env` file

☐ Secrets file created and secured (`.env` not committed to git)

☐ `application-secrets.yaml` created with your own `User-Agent` and `X-api-key` (see E-Rezept Fachdienst Credentials (`application-secrets.yaml`))

☐ Network connectivity the E-Rezept Fachdienst verified

☐ TLS certificates prepared (for production)

☐ Database volumes initialized

☐ Sufficient disk space available

☐ All build artifacts up-to-date

9.6.2. Local Development Workflow

This workflow is intended for local development and testing only; it is not suitable for production environments.

To reduce OAuth2 complexity during development, the local stack runs without Keycloak and without the OAuth2 BFF. Authentication is therefore disabled in this setup.

Contrary to the two possible Deployment Views the local Development Workflow is mostly the full deployment stack without the authentication server and the BFF. The local development workflow is intended for developers to quickly test changes in the backend or frontend without having to deal with the complexity of OAuth2 and Keycloak.



The `erpione-server` artifact must be built and repackaged before deployment; see Building Java/Kotlin Modules for the required `./mvnw package spring-boot:repackage -f erpione-server/pom.xml -DskipTests` step.

```
#!/bin/bash
set -e

ensure_volume() {
  local volume_name="$1"
  if ! docker volume inspect "$volume_name" >/dev/null 2>&1; then
    docker volume create "$volume_name" >/dev/null
    echo "Created volume: $volume_name"
  else
    echo "Volume already exists: $volume_name"
  fi
}

cd $(git rev-parse --show-toplevel)

# 1. Set environment
export ENVIRONMENT=rudev
export ERP_FHIR_PROFILE=1.6.0
export ACTIVE_SPRING_PROFILES=${ENVIRONMENT},noauth
export PUBLIC_PORT=8080

# 2. Create required Docker volumes
ensure_volume "erpione-data-${ENVIRONMENT}"

# 3. Build and start containers
docker compose -f dc-erpione-local.yml up -d --build

# 4. Wait for services to be healthy
echo "Waiting for services to start..."
sleep 15

# 5. Verify health
docker compose -f dc-erpione-local.yml ps
```

Now you can access the local development stack at `localhost:8080`. The REST API is available at `localhost:8080/api/info`.

9.6.3. Production Deployment

TODO: Document the following:

1. Pre-production checklist (security hardening, TLS setup, etc.)
2. Health check procedures
3. Load balancing setup (if applicable)
4. Backup and recovery procedures
5. Monitoring setup
6. Logging aggregation
7. Rollback procedures

Deployment Execution

```
#!/bin/bash
set -e

export ENVIRONMENT=rudev
export ERP_FHIR_PROFILE=1.6.0
export ACTIVE_SPRING_PROFILES=rudev,noauth

# Create volumes
for vol in erpione-data erpione-redis-data erpione-keycloak; do
  docker volume create "${vol}-${ENVIRONMENT}" || true
done

# Load secrets from external source
#TODO: Document secret injection mechanism

# Deploy
docker compose -f dc-erpione.yml up -d --build

# Monitor
docker compose -f dc-erpione.yml logs -f
```

9.7. Health Checks & Validation

After deployment, verify all services are operational:

9.7.1. Service Health Endpoints

Service	Endpoint	Expected Response
Resource Server	`localhost:8080/actuator/health`	`{"status":"UP"}`
BFF	`localhost:8080/actuator/health`	`{"status":"UP"}`
Reverse Proxy	`localhost:8080`	Nginx welcome page or proxied response
Keycloak	`localhost:8080/kc/health/ready`	HTTP 200 OK

Service	Endpoint	Expected Response
Redis	`redis-cli ping`	PONG

9.7.2. Manual Health Check Script

```
#!/bin/bash

echo "=== Erpione Health Check ==="

# Function to check HTTP endpoint
check_http() {
  local endpoint=$1
  local name=$2
  echo -n "Checking $name ($endpoint)... "
  if curl -s -f "$endpoint" > /dev/null; then
    echo "✓ OK"
  else
    echo "✗ FAILED"
    return 1
  fi
}

# Function to check container
check_container() {
  local container=$1
  echo -n "Checking container $container... "
  if docker ps --filter "name=$container" --filter "status=running" | grep -q "$container"; then
    echo "✓ Running"
  else
    echo "✗ Not running"
    return 1
  fi
}

check_http "http://localhost:8080/actuator/health" "Resource Server"
check_http "http://localhost:8080/kc/health/ready" "Keycloak"

check_container "erpione-resource-server"
check_container "erpione-bff"
check_container "erpione-redis"
check_container "erpione-keycloak"
check_container "erpione-reverse-proxy"

echo "=== Health Check Complete ==="
```

9.7.3. Database Verification

TODO: Document:

- How to verify database initialization
- How to check data persistence
- How to backup/restore database

```
# TODO: Add database health check commands
# Check H2 database connectivity
# Verify database schema initialization
```

9.7.4. API Verification

TODO: Document:

- How to test basic API endpoints
- How to create test data
- How to verify E-Rezept connectivity

```
# TODO: Add example API test commands
# Example: curl http://localhost:8080/api/v1/health
```

9.8. Logging & Monitoring

9.8.1. Application Logs

View container logs in real-time:

```
# All services
docker compose -f dc-erpione-local.yml logs -f

# Specific service
docker compose -f dc-erpione-local.yml logs -f resource-server

# With timestamps
docker compose -f dc-erpione-local.yml logs -f --timestamps

# Last N lines
docker compose -f dc-erpione-local.yml logs --tail=100 resource-server
```

9.8.2. Log Locations

Component	Log Location
erpione-server	Container stdout/stderr (docker logs) TODO: Document file-based logging location if applicable
erpione-bff	Container stdout/stderr (docker logs)
Keycloak	Container stdout/stderr (docker logs)
Nginx (reverse proxy & UI)	`/var/log/nginx/access.log` (inside container)

9.8.3. Metrics & Monitoring

TODO: Document the following:

1. Which metrics are exposed by each service?
2. How to scrape Prometheus metrics?
3. How to set up monitoring dashboards (Grafana)?
4. What alerts should be configured?

```
# TODO: Metrics endpoints

# erpione-server: http://localhost:8080/actuator/metrics
# erpione-bff: http://localhost:8080/actuator/metrics
# keycloak: TODO

# TODO: Example Prometheus scrape config
# TODO: Example Grafana dashboard setup
```

9.9. Troubleshooting

9.9.1. Common Issues

Services Not Starting

```
# Check Docker daemon status
docker version

# Check service status
docker compose -f dc-erpione-local.yml ps

# View service logs
docker compose -f dc-erpione-local.yml logs resource-server

# Common causes:
# - Port already in use: change PUBLIC_PORT in .env
# - Volume permission issues: check docker volume permissions
# - Out of memory: increase Docker memory allocation
```

Database Connection Failed

TODO: Document:

- Database connectivity troubleshooting
- Schema initialization issues
- Data corruption recovery

Authentication/Keycloak Issues

TODO: Document:

- OAuth2 client configuration errors
- JWT validation failures
- Session management problems
- Keycloak realm setup

Proxy/Network Issues

TODO: Document:

- SZZP/proxy connectivity errors
- TLS certificate validation failures

- DNS resolution issues
- E-Rezept backend connectivity

Appendix A: Release Notes

Release 1.0.0

Initial public release